

FOR DELVIN • 2026

私人工程教程 • 第二冊

AI 交易工程地圖

怎麼把 AI 接進交易流程 • 從資料到下單的全貌
一本寫給「進投資公司、要學讓 AI 幫忙交易」的你的工程入門

編纂：你的 AI 工程家教

對象：健康科學背景、即將在投資公司負責「AI 輔助交易」工程面的你

立場：誠實第一 • AI 是研究助理不是印鈔機 • 所有框架/論文/數字皆經網路查證（2025–2026）

完成日：2026 年 6 月 • 接《金融進階聖經》之後讀

老師的話：先把一個幻覺戳破

你老闆要你學「怎麼讓 AI 幫他們交易」。在我們開始之前，我必須先誠實地告訴你一件可能跟你想像相反的事——這也是這整本書的靈魂：

截至 2025–2026，沒有任何公開、嚴謹的證據顯示「LLM 交易代理」能穩定打敗大盤。多份同儕審查研究（FINSABER、StockBench）反覆發現：把資料偏誤拿掉、用夠長夠廣的測試後，這些光鮮亮麗的 AI 交易代理，**連最笨的「買進並持有」都打不過。**

所以這本書不會教你「打造一個會自己賺錢的 AI 機器人」——那是行銷話術，會讓你賠錢。它教你的是真正有價值、也是你老闆真正需要的工程能力：

把 AI 放在對的位置——當「研究助理」，不是「交易員」。

讓它讀你讀不完的財報新聞、清洗髒資料、生成特徵、輔助寫程式、做事後檢討。**下單與風控的最終判斷，留給人 + 規則。**

這本書怎麼鋪：

- **第一部 交易系統的解剖：**一筆交易從資料到下單的九個階段，AI 能插在哪、每階段的坑。先有全貌，才知道 AI 該擺哪。
- **第二部 AI 在交易裡的真實角色：**誠實拆解 LLM / 傳統 ML / 強化學習各能做什麼、不能做什麼。最重要的是「LLM 真正務實的五個用途」。
- **第三部 回測與驗證（全書最重要）：**回測框架地圖 + 驗證的數學。為什麼「漂亮的回測」是最危險的謊言。
- **第四部 資料、執行與上線紀律：**資料陷阱、券商 API、為什麼散戶贏不了速度、為什麼策略上線一定會衰退。
- **第五部 落地：**對照你手上已有的工具（trading_bot / quant_lab / TradingAgents...），給你公司場景的務實建議與學習路線。

讀完這本，你第一天上班就能講出「AI 在交易裡該放哪、不該放哪」，而且講得比多數只看過行銷文的人更內行、更誠實。我們開始。

目錄

1. 導讀：兩條主線

① 交易是一條資料生產線 ② AI 在關節、不在神諭 | 頻率軸線

2. 第一部 交易系統的解剖（九階段生命週期）

資料 → 清洗 → 特徵 → 決策 → 回測 → 風控 → 執行 → 監控 → 再學習

3. 第二部 AI 在交易裡的真實角色

LLM 真正務實的 5 個用途（最重要）

LLM 交易代理框架（TradingAgents / FinRobot / FinMem）與「打假」研究

傳統 ML 與強化學習的角色

4. 第三部 回測與驗證（全書最重要）

回測框架地圖（向量化 vs 事件驅動）

★ 驗證的數學：walk-forward / 樣本量 / Deflated Sharpe / 平台 vs 尖峰

5. 第四部 資料、執行與上線工程紀律

資料三陷阱 | 券商 API 與紙上交易 | 為何贏不了 HFT | Alpha Decay

6. 第五部 落地：你的工具箱與學習路線

對照你已有的 stack | 公司場景建議 | 動手實作課預告

7. 結語：把這 8 條工程守則放心裡

導讀：先記住兩條主線

這本書千頭萬緒，但只要記住兩條主線，所有細節都能掛回去。

主線一：交易系統不是「一支會賺錢的程式」，是一條「資料生產線」。

資料從一端進來，經過十幾道工序，最後變成「下單動作 + 績效報表」。任一道工序錯，整條線的產出就是垃圾（garbage in, garbage out）。而且——「想策略」只佔工程量一小塊，「驗證策略不是運氣」才吃掉 80% 心力。

主線二：AI 的位置在「關節」，不在「神諭」。

AI 散佈在生產線的多個關節當輔助（讀新聞抽情緒、清資料、生特徵、寫程式、做歸因），而不是「叫它預測明天漲跌」。判斷 AI 用不用得上，看一條軸線——

頻率軸線（最實用的一把尺）：越高頻、越要即時預測價格 → AI/LLM 越沒用（延遲輸、數值弱、無 edge）。越低頻、越依賴讀懂大量文字 → AI/LLM 越有用。這條軸線幾乎決定一切：你想讓 AI 幫的那件事，落在軸線哪一端？

第一部 交易系統的解剖

把一筆交易從無到有拆成九個階段。每階段講三件事：**在做什麼** → **AI 可以插在哪** → **常見坑**。這九階段是一個閉環（第 9 段的學習要回流到第 3、4 段），不是一條直線。一個成熟團隊的差距，往往不在策略多聰明，而在這個迴圈轉得多快、多誠實。

階段 1 資料擷取 (Data Ingestion)

在做什麼：從券商/交易所/資料商把行情（K 線、逐筆）、基本面、另類資料抓進來，可能是歷史回補（batch）或即時串流（WebSocket）。

AI 插在哪：爬蟲與 API connector；**LLM 把非結構化來源（財報 PDF、新聞、法說會逐字稿）轉成結構化欄位**。

常見坑：資料源時區/時間戳不一致；免費源（如 yfinance）無預警斷線或改格式；把「事後修正過的資料」當成「當時看得到的資料」（point-in-time 問題，見第四部）。

階段 2 清洗與對齊 (Cleaning & Alignment)

在做什麼：補缺值、處理除權息調整、對齊不同資產時間軸、過濾異常跳價。

AI 插在哪：異常偵測標出可疑跳點；但**最終規則最好是確定性、可重現的**，別讓 AI 黑箱亂改資料。

常見坑：**未調整 vs 已調整價格混用**（除權息造成假跳空）；用「未來才知道的均值」去填過去缺值＝偷看未來（look-ahead）。

階段 3 特徵 / 訊號生成 (Feature Engineering)

在做什麼：把原始價量算成指標（MA、RSI、波動率）、跨資產比率、情緒分數，變成模型/規則的輸入。

AI 插在哪：**這是 LLM/ML 最常合理介入的地方**——新聞情緒分數、財報語氣分類、另類資料特徵。**特徵工程才是 alpha 的真正來源**，模型只是把特徵組合起來。

常見坑：特徵裡偷藏未來資訊（用當日收盤算的指標卻在盤中就決策）；特徵太多、樣本太少 → 必然過擬合。

階段 4 策略決策 (Strategy Logic)

在做什麼：把訊號轉成「進場/出場/不動」的明確決定。

AI 插在哪：規則引擎、ML 分類器、或多代理 LLM 辯論架構；但 LLM 直接做即時買賣風險高，較穩的用法是「輔助判斷＋人/規則把關」。

常見坑：把「相關」當「因果」；策略可調參數太多（degrees of freedom）＝替過擬合鋪路。

階段 5 回測驗證 (Backtesting)

在做什麼：用歷史資料模擬「如果當時這樣做會怎樣」，估算報酬、回撤、Sharpe。

AI 插在哪：參數掃描可平行化；但**AI 幫你掃越多參數，過擬合風險越高**——雙面刃。

常見坑：回測漂亮 ≠ 實盤會賺。這階段坑最多、最致命，獨立成**第三部**詳述。

階段 6 風險管理 / 部位大小 (Risk & Sizing)

在做什麼：決定「這次下多少」、設停損、控制總曝險、相關性與集中度。

AI 插在哪：部位大小公式（風險式 sizing、分數 Kelly）、投組最佳化；AI 適合做情境模擬與壓力測試。

常見坑：用全 Kelly（太激進，波動拖累吃光複利）；忽略部位間相關性（以為分散其實同向）；停損設在技術無意義的位置。

階段 7 下單執行 (Execution)

在做什麼：把決策轉成真實訂單送到券商/交易所，處理成交、部分成交、拒單。

AI 插在哪：智慧拆單、執行演算法（TWAP/VWAP）；但散戶層級**先求穩不求快**。

常見坑：回測假設「以收盤價成交、零成本」，實盤有滑價、手續費、買賣價差；以為自己能跟 HFT 拚速度（結構性辦不到，見第四部）。

階段 8 上線監控 (Monitoring & Ops)

在做什麼：盯系統是否正常、訊號是否漂移、實盤績效是否偏離回測。

AI 插在哪：異常告警、自動 kill switch、AI 輔助寫值班報告。

常見坑：沒有監控就上線（出事才知道）；沒有 kill switch（策略崩潰時無法緊急停）。

階段 9 績效歸因 / 再學習 (Attribution & Feedback)

在做什麼：拆解「賺賠來自哪」、記錄每筆訊號的實際結果、回饋修正。

AI 插在哪：LLM 做交易事後檢討（postmortem）的文字總結與分類；建立「訊號 → 實際結果」的學習迴圈。

常見坑：只記得贏的、忘記輸的（選擇性記憶）；把「市場整體上漲（beta）」誤當成「策略很厲害（alpha）」。

第二部 AI 在交易裡的真實角色

這一部誠實拆解 AI 各路線能做什麼、不能做什麼。核心心法一句話：LLM 的 edge 在「把人類讀不完的文字，快速變成可用資訊」，不在「預測價格」。

★ LLM 真正務實的 5 個用途（本書最該抄走的一頁）

1. **新聞/財報/法說會的摘要與情緒分析**。把長篇逐字稿壓成重點＋語氣分數，跨全市場聚合找產業趨勢或異常公司。這是 LLM 投產 ROI 最高、最安全的用途。**產出「結構化摘要＋情緒分數」當特徵，下游決策另外做。**
2. **RAG 接財報/SEC/研究報告做問答**。先檢索原始文件片段，再讓 LLM 基於「檢索到的真實段落」回答、附出處。RAG 是目前對抗幻覺最實際的手段。**鐵律：所有數字必須能點回原文出處。**
3. **把非結構化文字轉成結構化訊號**。用 LLM 把「一段新聞」變成「利多/利空、影響哪檔、信心多少」的 JSON，餵給傳統量化模型。把 LLM 當「昂貴但很懂語意的標註器」。
4. **研究助理（產假設、寫程式、解釋因子）**。讓 AI 做「研究的工」：寫回測程式、解釋因子的經濟直覺、產候選假設、debug。**你的工作從「想點子」變成「嚴格證偽點子」。**
5. **事後檢討與績效歸因的文字總結**。把每筆交易的脈絡與結果讓 LLM 整理成 postmortem，餵回學習閉環。

反過來，LLM 不該做的：① 直接做高頻/即時價格預測（延遲毫秒~秒級，HFT 是微秒級，物理上輸）。② 自己算數字（decoder-only 架構數值推理天生弱，**數字一律交給程式**）。③ 「讀完財報直接告訴你買賣」（幻覺重災區）。**頻率越高、越要算數字，LLM 越沒用。**

LLM 交易代理框架：當架構教材，別信它的績效

這些是真實存在、值得 clone 下來讀程式的開源專案——但它們的「報酬數字」不能當投資依據：

多代理辯論

TradingAgents (arXiv 2412.20138)

模仿一家交易公司：基本面/情緒/技術分析師＋**多方 vs 空方研究員辯論**＋交易員整合＋風控團隊。**價值＝極好的「多代理工程架構教材」**（怎麼分角色、怎麼讓 agent 辯論、怎麼接資料源、怎麼加風控閘門）。

存疑：回測窗口極短（資料 2024 Q1、交易 2024 下半年）、標的少、摘要層級沒公布具體數字；且 2024 行情 LLM 訓練資料可能已涵蓋＝「回憶」而非「預測」。**讀架構，別信數字。**

FinRobot / FinGPT (AI4Finance 基金會)

FinGPT=開源金融 LLM，強在**結構化分類任務**（情緒分類、命名實體）。**FinRobot**=把 LLM+量化+工作流串起來的 AI agent 平台，適合當「自動產投研報告」的基礎建設。

存疑：2025 評估論文 (arXiv 2507.08015) 指出 FinGPT 在複雜推理、**數值運算**、關係抽取、摘要有明顯缺陷。也就是「會做文字分類，但算數字會出錯」——而交易恰恰要算數字。**把它當便宜的情緒標註器，數值計算交給 pandas。**

FinMem / FinAgent

強調「分層記憶+角色人格」的 LLM 交易代理 (FinAgent 再加 K 線圖的多模態)。「記憶+反思」是好的 agent 工程概念，值得學 (用在研究助理場景)。**但不要相信它原論文交易的績效**——那是窄窗口+精選標的的產物，正是下面「打假」研究的箭靶。

核心問題：這些 AI 代理到底能不能賺錢？（誠實版）

業界與論文的誠實結論：目前不能可靠地賺錢。三份 2025–2026「打假」研究是你的必讀疫苗：

研究	設定	結論
FINSABER (2505.07078, KDD 2026)	2000–2024、100+ 標的、去除三大偏誤	LLM 策略優勢「蒸發」；Buy-and-Hold 幾乎都排前段；配對 t 檢定無統計顯著 alpha (所有 p 值 > 0.34)。「目前 LLM 在現實中沒有打敗效率市場假說」
StockBench (2510.02209)	用「訓練後」的 2025 新資料避免污染，測 GPT-5 / Claude-4 / Qwen3	多數 LLM agent 連 Buy-and-Hold 都打不過。 「靜態金融問答拿高分，不等於會交易賺錢」
共同發現	多空 regime	LLM 有強烈 多頭偏誤 ：牛市過度保守（不敢賺）、熊市過度激進（賠更多）

老師補一句（接你自己的經驗）：這跟你 trading_bot 已經學到的硬教訓一模一樣——當年號稱 Sharpe 1.58 的 pair trade、+39%/年的 funding spike，用 Deflated Sharpe 一驗全是 cherry-pick 的運氣。**不管是傳統策略還是 LLM 代理，騙人的方式都一樣：窄窗口+挑最好的那組+忘了算「你試了幾次」。**你已經吃過這個虧，這是你比多數新人值錢的地方。

傳統 ML 與強化學習：別被 LLM 熱潮蓋過

監督式學習做訊號：用歷史特徵預測未來報酬，樹模型（XGBoost/LightGBM）至今仍是業界主力，在中低頻、多因子選股上經得起驗證、可解釋、成本低。**先學會樹模型+嚴謹交叉驗證，比學 LLM 交易代理更實用。**

時間序列：GARCH 在**波動度預測**（風控、選擇權）有真用途。但 FINSABER 發現——**更大更複雜的深度模型常贏不過簡單的 ARIMA**。複雜度不等於獲利；複雜模型必須證明它「顯著」打敗簡單基準才有資格上場。

強化學習（RL）——現實很骨感：最務實的場景是**最佳執行**（已知要買，怎麼切單買得便宜），而不是**決策**（該不該買）。困難在：市場非平穩（分布一直變）、Sim-to-Real gap（你的下單會改變市場，簡單模擬器假設不會）、訓練不穩。

新人最常見的幻想：把 RL 當「神級擇時機」。**RL 值得碰的是「執行層」（怎麼買），不是「決策層」（該不該買）。**

第三部 回測與驗證（全書最重要）

前面所有東西做得再好，如果你無法分辨「這個 edge 是真的，還是我在歷史雜訊裡挖到的巧合」，你就是拿真錢賭一個幻覺。**回測會說謊，而且它很會說謊。**這一部就是測謊機。

回測框架地圖：先懂一個分水嶺

理解所有框架的鑰匙，是**向量化 vs 事件驅動**：

	向量化（vectorized）	事件驅動（event-driven）
怎麼跑	把整段歷史當大矩陣，一次算完	像放電影，一格一格往前播，每步才決策
快不快	極快，適合掃幾千組參數	慢得多，但貼近真實交易
真不真	容易不小心偷看未來、難模擬複雜下單	天生強制「只能看當下」，易模擬滑價/部分成交
適合	研究、找訊號、大規模參數掃描	接近上線、驗證執行真實性

成熟 workflow 是兩者分工：用向量化引擎快速探索與篩選 → 用事件驅動引擎做真實重播與上線前驗證。**向量化不是「比較高級」，它只是「快」，真實性反而常輸事件驅動。**

各框架定位（Python 生態，皆查證 2025–2026 仍活躍）

框架	類型	定位 / 適合誰	注意
vectorbt	向量化之王	海量參數掃描、大範圍研究、機率分佈式穩健度測試	學習曲線陡；最強功能在付費 PRO 版
backtrader	事件驅動經典	散戶、學習、中低頻；建立「逐 bar 決策」直覺最佳入門	原作者已少維護（穩定但少更新，社群仍廣用）
zipline-reloaded	事件驅動	美股日頻、ML 研究；《ML for Algo Trading》指定引擎（2025 出 3.1.1，活躍）	偏美股日線、資料 bundle 設定繁瑣
nautilus_trader	工程級事件驅動	Rust 核心、奈秒級、 回測碼直接上實盤不用重寫 ；要真上線的進階者	門檻高，對純研究階段偏重
QuantConnect/LEAN	引擎+雲端	「回測→實盤」一站式、多資產、整合 20+ 券商	深用雲端要付費；策略放別人平台的疑慮
bt	輕量投組	資產配置、再平衡、權重型策略	不適合精細進出場/執行模擬

新人常見誤解：「找到最強框架就贏一半」——錯，**框架只是計算機，賺不賺錢取決於你餵進去的想法與驗證紀律。** | 「回測賺很多＝策略好」——錯，回測是最容易自欺的環節。

★ 驗證的數學：整本書的靈魂（只要懂四個概念）

① 走查驗證 (Walk-Forward)

不要只把資料切成「前面訓練/後面測試」一次。改成滾動窗口：訓練一段 → 測試緊接的下一段 → 視窗往前滑 → 再訓練再測試……反覆做。**強迫策略在多個不同時期、多種市場狀態反覆證明自己**，而不是靠一段幸運期。它是業界最被接受的驗證方式。

② 樣本量 $N \approx 1 / \text{edge}^2$ (直覺很重要)

你的 edge (優勢) 越小，需要越多筆交易才能在統計上確認它是真的。

edge (優勢)	大約需要的交易筆數
10% (0.1)	約 100 筆
5%	約 400 筆
1%	約 10,000 筆

為什麼致命：新人常拿「20 筆交易勝率很高」就信心爆棚——**20 筆什麼都證明不了**，純運氣就能造出漂亮的小樣本。沒有足夠樣本，任何漂亮數字都不算數。

③ 多重檢定 / Deflated Sharpe Ratio (DSR)

問題：你試了越多組參數/越多策略，「總會有一組剛好看起來很神」——純靠運氣。報告那一組，就是在自欺。

DSR 是什麼：Bailey & López de Prado (2014) 提出，把你「試了幾次」、報酬分佈的偏態峰態、樣本長度都納入，**回頭把你宣稱的 Sharpe「打折」**，給出更誠實的顯著性。

白話結論：你掃描的參數越多，你那個最佳 Sharpe 就越該被懷疑。新人鐵律——**記錄你跑過幾次回測**。(這正是你 CLAUDE.md 裡 backtest-validation / quant-math 技能在做的事。)

④ 平台型 vs 尖峰型參數 (穩健度的視覺直覺)

把「參數 vs 報酬」畫成一座山：

- **平台型 (plateau)：**一整片參數都還不錯，中間沒掉下去 → edge 穩健，參數抓不完美也還行，**像是真的**。
- **尖峰型 (spike)：**只有某個極窄參數值超神，旁邊一點點就崩 → 你**剛好擬合到歷史雜訊**，實盤幾乎一定失效。

實務鐵則：寧可選平台中間一個普通的點，也不要選那根孤立的尖峰。看到尖峰就該警鈴大作。

本章靈魂 (全書最重要的一句)：驗證不是「最後蓋章的手續」，而是用來「殺掉」絕大多數其實沒用的策略的篩子。一個健康的研究流程，**99% 的點子都該死在這一關**。能誠實砍掉自己心血的人，才做得了量化。

第四部 資料、執行與上線工程紀律

資料工程：品質決定上限，策略只決定你離上限多近

新人最容易低估這章，但專業團隊一半的工在這裡。常用行情源（價格/額度 2025–2026 會變動，實作前再查官網）：

來源	特性	注意
yfinance (Yahoo)	免費無金鑰、入門極友善	非官方無 SLA，會斷線/改格式；只適合學習與原型
FinMind	台股為主、含基本面與法人	台股研究常用；免費層有額度（你既有專案已在用）
Finnhub	免費層大方、即時行情+新聞/情緒	適合做情緒特徵
Polygon.io	WebSocket 串流、分鐘級強	付費（\$199/月起）；偶有資料可信度個別評論
Interactive Brokers	券商等級行情+可下單	需開戶；API 有速率限制

三個會讓回測「假賺」的資料陷阱（必記）：

- **倖存者偏誤**：資料庫只留「現在還活著」的公司，下市/破產的被刪 → 等於只挑贏家，報酬每年可能虛高數個百分點。解法：用含已下市標的的資料庫。
- **未來函數 (look-ahead)**：回測偷用了當時還沒公布的資料（例如財報實際公布日 vs 財報期末日的落差）。
- **Point-in-Time (時點正確)**：財報常被事後重編、經濟指標常被修正。**回測必須用「當時版本」的資料，不是「現在版本」。**這是專業與業餘資料工程最大的分水嶺。

下單執行：先求穩，不求快

主要管道：CCXT（加密貨幣統一函式庫，一套 API 接 100+ 交易所，[做加密的事實標準](#)）、Interactive Brokers（傳統市場主力）、Alpaca（美股）、台股本土券商（如 Shioaji）。

紙上交易 (paper trading) 是必經之路。它是「回測」與「真錢」之間**必須**的中間層，會暴露回測永遠看不到的東西：真實滑價、延遲、部分成交、拒單、API 斷線、你自己程式的 bug。

標準漸進路徑：回測 → 紙上交易 → 小資金實盤 → 逐步放大。（你的 trading_bot 預設 paper 模式就是對的。）

maker vs taker：taker（市價吃單）付較高費+吃價差+滑價；maker（掛限價）費低甚至有回饋，但**不保證成交**。maker 是「有條件的成本優勢」，不是必勝。

為什麼散戶贏不了延遲競賽（結構性，不是你不夠努力）：真正的 HFT 是**微秒級**（主機共置、微波/光纖專線、FPGA）；散戶用 API 是**10–50 毫秒級**+下單速率上限，慢上千倍。**結論：不該玩「比誰快」的遊戲。**你的 edge 只能來自別人沒看到的角度、沒耐心等的時間框架、懶得做的資料工——不是速度。

上線工程紀律 (MLOps)：交易的 bug 不是當機，是虧損

- **版本控制與可重現性**：不只程式進 Git——資料版本、參數、隨機種子、套件版本都要鎖。目標：**任何一次回測/交易，半年後都能一字不差重跑出同樣結果**。做不到重現，就無法判斷「績效變差」是市場變了還是你改壞了。

- **漸進式上線**：每一階都待夠久、收集足夠樣本才升級。每階問同一個問題：「下一階暴露的新現實（成本/延遲/容量/自己的心理），會不會殺死這個 edge？」
- **監控兩種健康**：系統健康（API 通不通、資料有沒有進來、程式有沒有掛）＋策略健康（實盤 vs 回測是否偏離、訊號分佈有沒有漂移）。出狀況主動推播告警。
- **Kill switch / 風控閘門**：一鍵/自動緊急停止（單日虧損超標、連虧 N 筆）；上線前硬門檻（如 $DSR < 0.95$ 不准上）。設計原則 **fail-closed**（出錯預設「停止」而非「繼續亂跑」）。

Alpha Decay：新人最震撼的真相——好策略上線後，報酬會自然衰減，這是常態不是意外。原因：市場結構改變、過度擁擠（很多人用同樣因子）、訊號加速腐壞（可能在你下單前就過期）。

所以能持續賺的，是「能持續找到並汰換 edge 的系統與紀律」，不是某條神奇規則。沒有聖杯策略可以退休。

第五部 落地：你的工具箱與學習路線

好消息是——你不是從零開始。你手上已經有一整套可以直接拿來練的資產。這一部把「地圖」對應到「你已有的東西」，並給你一條學習路線。

你已經有的 stack（對照地圖看）

地圖環節	你已有的工具 / 技能
資料源	FinMind（台股+美股+TAIEX）、Finnhub（MarketDaily 在用）、yfinance fallback
特徵 / 訊號（LLM 路線）	MarketDaily 整套 AI 管線（新聞點名個股、情緒、財報摘要）、Grok 政壇訊號
策略 / 多代理	tradingagents 技能、finrobot 技能、edge-pipeline-orchestrator
回測 / 驗證	backtest-validation（walk-forward / DSR）、quant-math、trading_bot 的 analyze_robustness.py
風控 / 部位	position-sizer、quant-math（Kelly/波動拖累）、trading_bot 風控閘門
執行	ccxt 技能（100+ 交易所）、trading_bot（paper 模式）、order-execution
上線 / 監控	trading_bot supervisor、winrig 24h 主機、告警通道
再學習	signal-postmortem、auto_tune 自學系統

老師的觀察：你缺的不是工具，是**把它們串成一條「誠實閉環」並養成驗證紀律**。你過去在 trading_bot 上踩過的雷（假 Sharpe、cherry-pick），其實已經幫你學會這本書最難的第三部了——多數人是賠了真錢才懂。

給你公司場景（台股 + 量子主題投資）的務實建議

1. 先做「AI 研究助理」，不要碰「AI 自動交易」。對老闆最快見效、風險最低的，是把 LLM 用在第二部那五件事——台股財報/法說會摘要、新聞情緒、公司文件 RAG 問答。這直接幫老闆省下讀資料的時間，且不會賠錢。
2. 量子主題正好是 RAG 的好題目。把台股量子概念股的財報、法說會、技術新聞餵進 RAG，做「這家是真布局還是蹭題材」的問答助理（你的 quantum_tracker 已有雛形）。
3. 任何「訊號」上線前，先過第三部的驗證篩子。walk-forward + DSR + 含下市股 + 真實成本。先打敗 Buy-and-Hold 與 ARIMA 再說。
4. 數字交給程式，文字交給 LLM。建 guardrail：LLM 輸出的任何數值都要能點回來源、跟合理範圍對賬。

學習路線（四步，搭配動手實作課）

① 讀懂全貌（這本書）→ ② 練資料+回測（vectorbt 或 backtrader 跑通一個簡單策略，學會 walk-forward）→ ③ 練 LLM 研究助理（做一個台股財報 RAG 問答 + 新聞情緒特徵）→ ④ 串成閉環（訊號 → 回測驗證 → paper → 監控 → postmortem）。

下一冊預告（動手實作課）：我會帶你用你現有的工具，一步步跑通一個最小可動 demo——「AI 給訊號 → 回測驗證 → 風控 → paper 下單」的完整閉環。不是玩具，是能展示給老闆看的真東西。

結語：把這 8 條工程守則放心裡

1. **AI = 研究助理，不是交易員**。讓它讀文件、寫程式、產假設、做摘要；下單與風控留給人 + 規則。
2. **頻率軸線決定一切**：越高頻越沒用，越依賴讀文字越有用。
3. **LLM 最高 ROI 三件事**：財報/法說會摘要與情緒、RAG 文件問答（綁出處）、文字→結構化特徵。
4. **數字交給程式，不交給 LLM**。金融幻覺是會賠錢的。
5. **任何策略上線前必過去偏誤檢查**：樣本外、walk-forward、Deflated Sharpe、point-in-time、含下市股、真實成本。記得「你試了幾次回測」。
6. **先打敗最笨的基準**：先穩定贏過 Buy-and-Hold 與 ARIMA，再談複雜化。多數論文這關就掛了。
7. **讀 repo 學工程，別信 repo 的績效數字**。TradingAgents/FinRobot 是好架構教材，不是賺錢保證。
8. **沒有聖杯，只有閉環**。Alpha 會腐壞；能持續賺的是「持續找到並汰換 edge 的系統與紀律」。

你進這家公司的真正價值，不是「會用最炫的 AI」，而是「**知道 AI 該放哪、不該放哪，而且有紀律去證明一個 edge 是不是真的**」。這正是這本書教你的，也是你 trading_bot 的失敗早就替你上過的課。下一冊我們動手做。慢慢來，比較快。